

Báo cáo đồ án cuối khóa

Python cho Khoa học dữ liệu / Dự đoán khối lượng chất béo

Mục lục

1	Giới thiệu bài toán và dữ liệu	2
1.1	Mô tả dữ liệu và EDA	2
1.2	Phân tích dữ liệu	6
1.3	Ý nghĩa thực tiễn của bài toán	6
1.3.1	Đối với người nội trợ	6
1.3.2	Đối với người ăn kiêng và người tập luyện	6
1.3.3	Đối với người bình thường có nhu cầu theo dõi sức khỏe	7
1.3.4	Ứng dụng trong tương lai	7
2	Giới thiệu các Class và phương thức trong project	7
2.1	Package: src.data	7
2.1.1	src/data/io.py	7
2.1.2	src/data/transformer.py	8
2.1.3	src/data/preprocessor.py	9
2.2	Package: src.models	10
2.2.1	src/models/evaluator.py	10
2.2.2	src/models/io.py	11
2.2.3	src/models/trainer.py	11
2.2.4	src/models/optimizer.py	12
2.3	Package: src.visualization	13
2.3.1	src/visualization/eda.py	13
2.3.2	src/visualization/model_plots.py	14
3	Tổng quan thư viện và phương pháp tối ưu siêu tham số	14
3.1	Thư viện học máy sử dụng	14
3.2	Các mô hình được sử dụng	14
3.3	Chỉ số đánh giá mô hình	15
3.4	Tối ưu siêu tham số bằng phương pháp Bayesian	15
4	Kết quả thực nghiệm	15
4.1	So sánh các mô hình học máy	16
4.2	Phân tích đặc trưng quan trọng của các mô hình	17
5	Bảng công việc	19

1 Giới thiệu bài toán và dữ liệu

Trong project này, bài toán được lựa chọn là **hồi quy (regression)**, với mục tiêu dự đoán **khối lượng chất béo bão hòa của các món ăn nhanh** tại những chuỗi cửa hàng lớn, dựa trên các đặc trưng dinh dưỡng cơ bản, dễ dàng tính toán được của từng sản phẩm.

Bộ dữ liệu sử dụng là *Fast_Food_Dataset*, được công bố trên Kaggle¹.

1.1 Mô tả dữ liệu và EDA

Dưới đây là mô tả chi tiết các trường dữ liệu có trong bộ dữ liệu, được thể hiện trong Bảng 1.

Cột	Mô tả
Company	Tên công ty hoặc thương hiệu cung cấp sản phẩm.
Item	Tên món ăn hoặc sản phẩm cụ thể.
Calories	Tổng lượng calo trong một khẩu phần.
Calories from Fat	Lượng calo có nguồn gốc từ chất béo.
Total Fat (g)	Tổng lượng chất béo (gram) trong một khẩu phần.
Saturated Fat (g)	Lượng chất béo bão hòa (gram).
Trans Fat (g)	Lượng chất béo chuyển hóa (gram).
Cholesterol (mg)	Hàm lượng cholesterol (mg).
Sodium (mg)	Hàm lượng natri/muối (mg).
Carbs (g)	Tổng lượng carbohydrate (gram).
Fiber (g)	Hàm lượng chất xơ (gram).
Sugars (g)	Lượng đường (gram).
Protein (g)	Hàm lượng protein (gram).
Weight Watchers Pnts	Điểm số theo hệ thống đánh giá dinh dưỡng Weight Watchers.

Bảng 1: Bảng mô tả dữ liệu Fast_Food_Dataset

Các cột được loại bỏ:

- **Company** và **Item**: Đây là các đặc trưng không mang bất cứ ý nghĩa nào trong bài toán.
- **Calories** và **Calories from Fat**: Hai đặc trưng này có mối quan hệ chặt chẽ với nhau.

$$\text{Calories from Fat} = \text{Total Fat (g)} \times 9$$

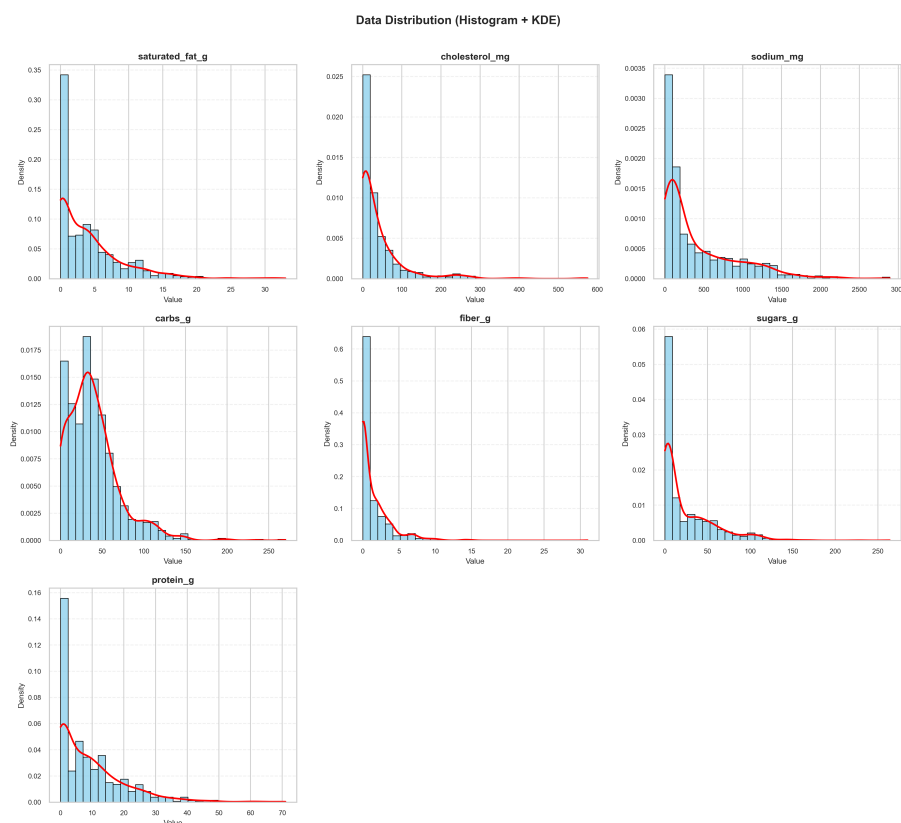
Và ngoài ra Calories có thể được tính như sau:

$$\text{Calories} = \text{Proten (g)} \times 4 + \text{Carbs (g)} \times 4 + \text{Total Fat (g)} \times 9$$

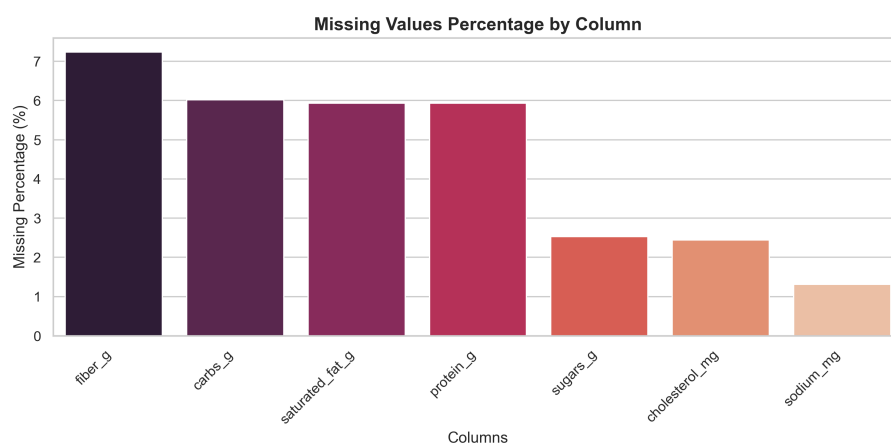
Vậy có thể thấy nếu giữ cả 2 cột này sẽ dễ dàng xảy ra hiện tượng đa cộng tuyến, cho ra kết quả không tốt.

¹<https://www.kaggle.com/datasets/tan5577/nutritonal-fast-food-dataset>

- **Total Fat (g)** và **Trans Fat (g)**: Hai đặc trưng này chỉ có thể đo chính xác bằng thiết bị chuyên dụng², ta chỉ có thể ước lượng với sai số lớn từ các bảng quy đổi trên mạng.
- **Weight Watchers Pnts**: Điểm số này được tính toán dựa trên **Calories** là chủ yếu, dữ liệu không mang nhiều ý nghĩa trong việc dự đoán chất béo bão hòa.

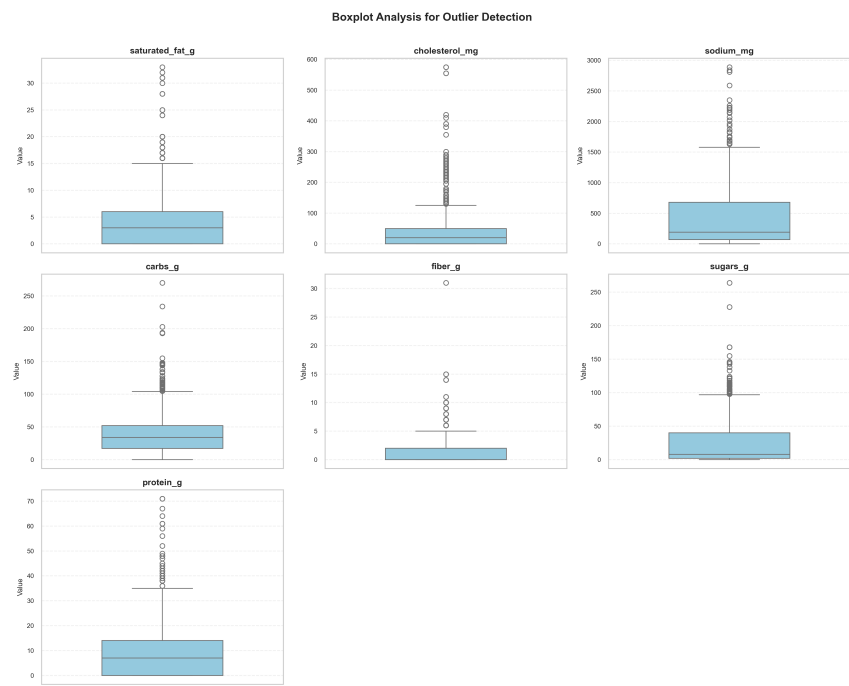


Hình 1: Phân phối dữ liệu các đặc trưng

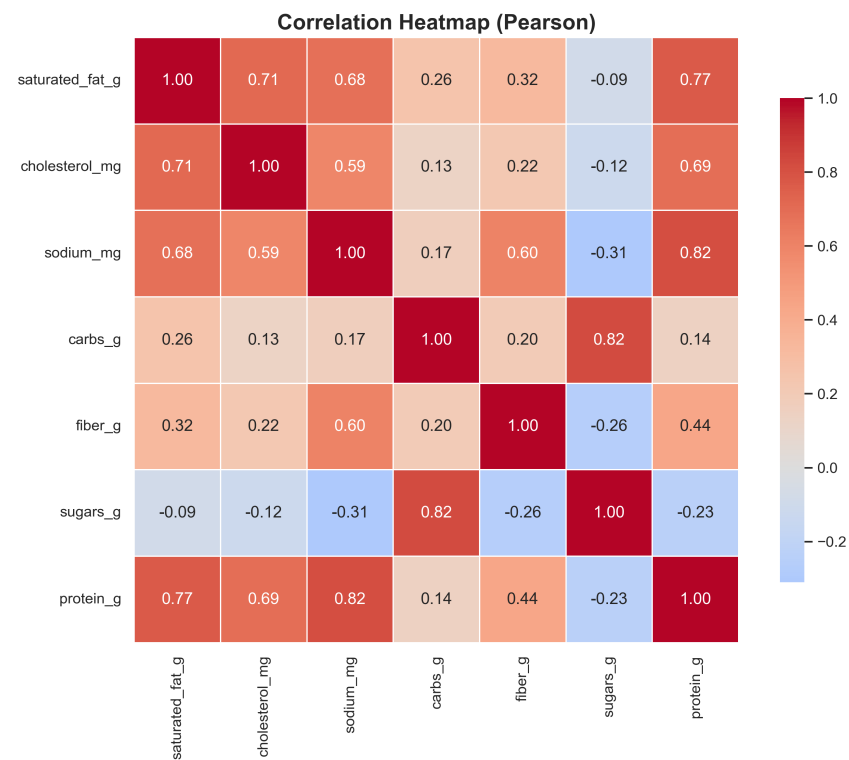


Hình 2: Biểu đồ giá trị khuyết trong dữ liệu

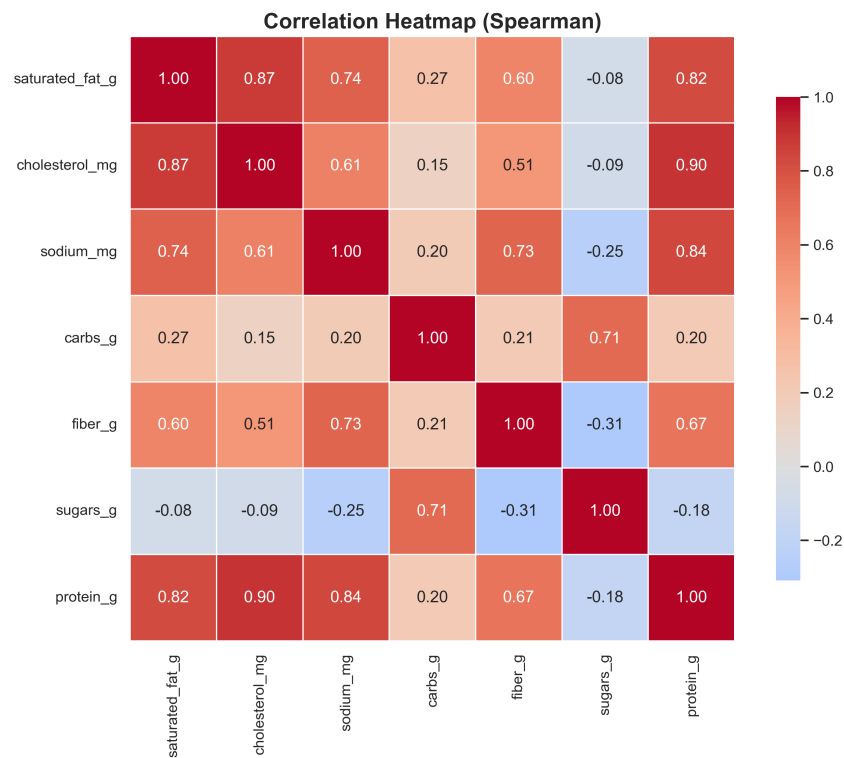
²Khác với Cholesterol và Sodium, ta có thể ước lượng quy đổi dễ dàng hơn do nó ít bị biến đổi khi chế biến, ngoài ra được ghi rất rõ trong các nhãn sản phẩm khi mua hàng



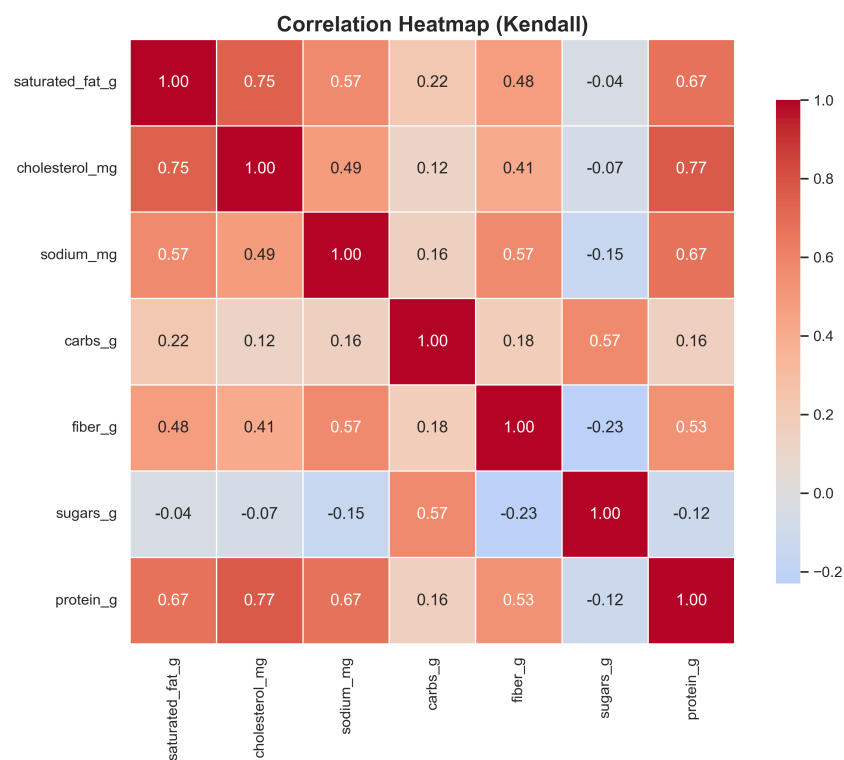
Hình 3: Biểu đồ hộp (boxplot) các đặc trưng



Hình 4: Heatmap tương quan các đặc trưng (Pearson)



Hình 5: Heatmap tương quan các đặc trưng (Spearman)



Hình 6: Heatmap tương quan các đặc trưng (Kendall)

1.2 Phân tích dữ liệu

Bộ dữ liệu này có phân phối lệch phải với toàn bộ đặc trưng, chứa khá nhiều giá trị ngoại lai (outliers), ta sẽ phải xem xét để quyết định áp dụng các phương pháp xử lý phù hợp.

Hình 2 minh họa số lượng giá trị khuyết trong từng đặc trưng, điều đáng chú ý là cột mục tiêu **saturated_fat_g** cũng có một lượng giá trị khuyết, ta sẽ phải bỏ các hàng bị thiếu này đi, ngoài ra khi xem xét kỹ hơn, các hàng này đều chứa giá trị thiếu ở hầu hết các đặc trưng của nó đây chính là lỗi nhập liệu.

Dựa trên heatmap tương quan ở Hình 4, Hình 5 và Hình 6, ta rút được một số nhận xét quan trọng sau:

- **cholesterol_mg**, **sodium_mg** và **protein_g** là các đặc trưng có tương quan cao nhất với mục tiêu.
- Đặc trưng **protein_g** có tương quan khá cao với các đặc trưng khác, điều này có thể dẫn tới đa cộng tuyến nếu không cẩn thận, ảnh hưởng tới hiệu năng mô hình.

Dựa trên ngữ nghĩa thực tế, ta sẽ **chuyển toàn bộ giá trị âm thành giá trị dương** (nếu có tại đây chính là lỗi nhập liệu—typo).

1.3 Ý nghĩa thực tiễn của bài toán

Việc ước lượng hàm lượng chất béo xấu (bao gồm *saturated fat* và *trans fat*) từ các chỉ số dinh dưỡng dễ thu thập như *cholesterol*, *sodium* (dựa vào nhãn thành phần), *carbohydrate*, *fiber*, *sugars* và *protein* (có thể cân dễ dàng ở nhà) mang lại nhiều ý nghĩa thực tiễn quan trọng trong đời sống hằng ngày, đặc biệt trong bối cảnh người dùng phổ thông không có điều kiện đo lường chính xác các thành phần chất béo bằng thiết bị chuyên dụng.

1.3.1 Đối với người nội trợ

- Ước lượng nhanh mức độ chất béo xấu trong bữa ăn hằng ngày.
- Điều chỉnh lượng gia vị, dầu mỡ và các thực phẩm có nguồn gốc động vật.
- Chủ động bảo vệ sức khỏe tim mạch cho các thành viên trong gia đình.

1.3.2 Đối với người ăn kiêng và người tập luyện

- Duy trì tỷ lệ mỡ cơ thể ở mức hợp lý.
- Hạn chế nguy cơ rối loạn mỡ máu trong quá trình ăn kiêng kéo dài.
- Tối ưu hoá hiệu quả tập luyện thông qua chế độ dinh dưỡng lành mạnh.

Mô hình dự đoán giúp nhóm đối tượng này kiểm soát chất béo một cách chủ động ngay cả khi không có sẵn thông tin chi tiết trên nhãn thực phẩm.

1.3.3 Đối với người bình thường có nhu cầu theo dõi sức khỏe

Đối với người trưởng thành bình thường, việc duy trì lượng *saturated fat* trong giới hạn khuyến nghị (dưới 10% tổng năng lượng mỗi ngày, lý tưởng dưới 7%) giúp giảm đáng kể nguy cơ mắc các bệnh tim mạch, cao huyết áp và tiểu đường type 2. Hệ thống ước lượng từ mô hình học máy có thể:

- Cảnh báo sớm khi lượng chất béo xấu vượt ngưỡng an toàn.
- Hỗ trợ người dùng điều chỉnh khẩu phần ăn hợp lý theo từng ngày.
- Hình thành thói quen ăn uống khoa học và bền vững.

1.3.4 Ứng dụng trong tương lai

- Thực hiện triển khai trên các ứng dụng điện thoại.
- Hỗ trợ tích hợp tính toán Cholesterol và Sodium nhanh hơn thay vì phải tìm kiếm nhập liệu thủ công.

2 Giới thiệu các Class và phương thức trong project

Phần này mô tả cấu trúc mã nguồn của project và giải thích **chức năng của từng Class cùng các phương thức (methods) chính**. Hệ thống được tổ chức trong thư mục `src/` và chia thành ba package chính: `src.data`, `src.models` và `src.visualization`. Cách trình bày được thiết kế theo phong cách tài liệu kỹ thuật, giúp người đọc dễ dàng tra cứu khi muốn xem lại luồng xử lý của chương trình.

2.1 Package: `src.data`

Package này chịu trách nhiệm xử lý dữ liệu đầu vào, bao gồm: đọc/ghi file, làm sạch dữ liệu thô, xử lý giá trị khuyết, ngoại lai và chuẩn hoá dữ liệu để đưa vào mô hình học máy.

2.1.1 `src/data/io.py`

Class: `DataIO`

Class chuyên trách việc Đọc/Ghi file và chuẩn hóa tên cột. Đảm bảo dữ liệu được nạp vào và lưu ra đúng định dạng, nhất quán với toàn bộ pipeline.

Methods:

`load_data(filepath)`

Nạp dữ liệu từ các file (CSV, XLSX, JSON) vào `pandas DataFrame`. Hỗ trợ xử lý lỗi khi file không tồn tại hoặc sai định dạng, giúp chương trình không bị crash đột ngột.

`save_data(data, filepath)`

Lưu `DataFrame` hiện tại vào file CSV tại đường dẫn chỉ định. Được dùng sau khi dữ liệu đã qua xử lý để lưu lại phiên bản *sạch*.

`clean_column_names(data)`

Chuẩn hóa tên cột: loại bỏ ký tự đặc biệt, khoảng trắng thừa và chuyển về dạng **snake_case** để thuận tiện cho việc code và tương thích với các hàm xử lý tiếp theo.

2.1.2 src/data/transformer.py**Class: DataTransformer**

Class chứa các thuật toán cốt lõi để xử lý dữ liệu như: xử lý giá trị thiếu, xử lý ngoại lai, chuẩn hóa, mã hóa, và chuyển đổi kiểu dữ liệu. **DataTransformer** hoạt động ở dạng các hàm thuần (pure functions/static methods) trên **DataFrame**.

Methods:`auto_detect_columns(data)`

Tự động phát hiện và phân loại các cột theo nhóm: *numeric*, *categorical*, *datetime*. Thông tin này được dùng để quyết định chiến lược tiền xử lý phù hợp cho từng loại cột.

`auto_convert_numeric_columns(data, threshold)`

Tự động rà soát các cột kiểu **object** và cố gắng chuyển sang kiểu số. Nếu tỉ lệ giá trị chuyển đổi thành công vượt **threshold** (mặc định 80%), cột đó sẽ được chuyển đổi thành kiểu số.

`convert_to_datetime(data, columns, date_format)`

Chuyển đổi các cột được chỉ định sang kiểu **datetime** theo định dạng ngày tháng cung cấp, hỗ trợ chuẩn hóa dữ liệu thời gian.

`clean_negative_values(data)`

Thay thế tất cả giá trị âm trong các cột số bằng giá trị tuyệt đối của chúng. Điều này bám sát quyết định xử lý ở Mục 1, vì các chỉ số dinh dưỡng không nên mang giá trị âm.

`handle_missing_values(data, strategies, learned_values, exclude_features, fit)`

Xử lý giá trị thiếu theo chiến lược cấu hình cho từng loại cột (numeric, categorical, datetime). Hỗ trợ chế độ **fit=True** để học tham số từ train set và **fit=False** để áp dụng cho test set, tránh *data leakage*.

`handle_outliers(data, method, exclude_features)`

Phát hiện và loại bỏ các hàng chứa giá trị ngoại lai trong cột số sử dụng các phương pháp như IQR, Z-score hoặc Isolation Forest, phù hợp với phần mô tả ở Hình 3.

`encode_categorical(data, strategy, encoders, fit)`

Mã hóa các biến phân loại thành dạng số để mô hình có thể sử dụng. Hỗ trợ Label Encoding và One-Hot Encoding với cơ chế fit/transform riêng biệt cho train/test set.

`scale_features(data, strategy, scaler, exclude_features, fit)`

Chuẩn hóa dữ liệu số về cùng một khoảng (sử dụng `StandardScaler` hoặc `RobustScaler`), khớp với các chiến lược scaling đã trình bày trong Bảng 2.

`remove_duplicates(data, subset)`

Kiểm tra và loại bỏ các hàng trùng lặp hoàn toàn hoặc trùng trên một tập cột con (`subset`), giúp giảm nhiễu.

`drop_null_targets(data, target_column)`

Loại bỏ các hàng có giá trị target null, đảm bảo dữ liệu huấn luyện không chứa nhãn thiếu.

`drop_features(data, features)`

Loại bỏ hoàn toàn các cột không cần thiết hoặc đã được xác định là gây ra *data leakage*.

2.1.3 src/data/preprocessor.py

Class: DataPreprocessor

Class đóng vai trò *orchestrator* (bộ điều phối), kết hợp `DataIO` và `DataTransformer` để tạo thành một pipeline xử lý dữ liệu hoàn chỉnh. Class này lưu giữ trạng thái của dữ liệu trong quá trình biến đổi và hỗ trợ method chaining.

Methods:

`__init__(num_strategy, cat_strategy, dt_strategy, scaling_strategy, outlier_method)`

Khởi tạo đối tượng với các tham số cấu hình cho toàn bộ quy trình: chiến lược xử lý missing values cho từng loại cột (numeric, categorical, datetime), phương pháp scaling và phương pháp xử lý outlier.

`load_data(filepath, auto_convert_numeric, auto_convert_threshold)`

Gọi `DataIO.load_data()` để nạp dữ liệu từ file, thực hiện chuẩn hóa tên cột và tự động chuyển đổi các cột có thể là số nếu `auto_convert_numeric=True`.

`save_data(filepath)`

Lưu trạng thái dữ liệu hiện tại (sau khi xử lý) xuống file CSV, giúp tái sử dụng hoặc chia sẻ cho các bước phân tích khác.

`auto_detect_columns()`

Phân loại các cột (numeric, categorical, datetime) và lưu thông tin này vào thuộc tính nội bộ của class.

`convert_to_datetime(columns, date_format)`

Gọi hàm chuyển đổi ngày tháng và cập nhật lại `DataFrame` nội bộ.

`clean_negative_values()`

Thực hiện làm sạch giá trị âm trên dữ liệu hiện có, đảm bảo tính hợp lý về mặt ngữ nghĩa dinh dưỡng.

`handle_missing_values(data, exclude_features, fit)`

Xử lý giá trị thiếu theo chiến lược đã cấu hình. Hỗ trợ chế độ `fit=True` (học tham số điền trên train set) và `fit=False` (áp dụng cùng tham số đó cho test set) để tránh *data leakage*.

`handle_outliers(data, exclude_features)`

Loại bỏ các hàng chứa giá trị ngoại lai dựa trên phương pháp đã được cấu hình (`outlier_method`).

`encode_categorical(data, strategy, fit)`

Mã hóa biến phân loại và lưu trữ các encoder để áp dụng nhất quán cho dữ liệu mới.

`scale_features(data, exclude_features, fit)`

Chuẩn hóa dữ liệu số, lưu lại `scaler` đã `fit` để dùng cho tập kiểm tra hoặc dữ liệu triển khai thực tế.

`remove_duplicates(subset)`

Loại bỏ các hàng trùng lặp trên dữ liệu hiện tại.

`drop_null_targets(target_column)`

Loại bỏ các hàng có giá trị target null, đảm bảo dữ liệu huấn luyện hợp lệ.

`drop_features(features_to_drop)`

Xóa các cột không còn cần thiết (ví dụ: `calories_from_fat`, `weight_watchers_pnts`, `company`, `item` như phân tích ở mục trước).

`get_processed_data()`

Trả về `DataFrame` kết quả sau khi đã trải qua toàn bộ pipeline tiền xử lý.

2.2 Package: `src.models`

Package này quản lý vòng đời của mô hình học máy: từ huấn luyện, tối ưu siêu tham số cho đến đánh giá và lưu mô hình.

2.2.1 `src/models/evaluator.py`

Class: ModelEvaluator

Class tiện ích (utility) cung cấp các phương thức tính để tính toán các chỉ số đánh giá hiệu quả mô hình.

Methods:

`evaluate_model(model, X_test, y_test, model_name)`

Dự đoán trên tập test và tính toán các metrics: MSE, RMSE, MAE, R^2 . Kết quả được trả về dưới dạng dict để dễ lưu trữ và phân tích.

`get_feature_importance(model, feature_names, model_name, top_n)`

Trích xuất độ quan trọng của các đặc trưng từ mô hình đã huấn luyện (hỗ trợ cả mô hình tuyến tính với `coef_` và tree-based với `feature_importances_`). Kết quả này liên kết trực tiếp với phần phân tích ở Hình 9.

2.2.2 src/models/io.py

Class: ModelIO

Class chịu trách nhiệm *serialization* (lưu trữ) và *deserialization* (khôi phục) các mô hình và kết quả đánh giá.

Methods:

`load_model(filepath, method)`

Đọc file mô hình từ ổ cứng và khôi phục thành object Python. Hỗ trợ các phương thức: 'joblib' (mặc định) và 'pickle'.

`save_model(model, model_name, filepath, method)`

Lưu object mô hình xuống file để tái sử dụng sau này. Nếu `filepath=None`, tự động tạo tên file với `timestamp` trong thư mục 'outputs/models/'.

`save_results(results, filepath, format)`

Xuất danh sách kết quả đánh giá (metrics) ra file CSV hoặc JSON, giúp liên kết với file Google Sheets trong Mục 4.

2.2.3 src/models/trainer.py

Class: ModelTrainer

Class trung tâm quản lý quy trình huấn luyện, kết nối giữa dữ liệu đã xử lý, các thuật toán mô hình và bước tối ưu siêu tham số. Việc đánh giá và IO được tách ra các module riêng (ModelEvaluator, ModelIO).

Attributes:

- `best_model_name`: Property trả về tên model có R^2 cao nhất từ `results`.
- `best_model`: Property trả về model object tốt nhất từ `trained_models`.

Methods:

`__init__(random_state, n_jobs)`

Khởi tạo Trainer với `random_state` cố định để đảm bảo tính tái lập (*reproducibility*) và số jobs song song `n_jobs`.

`load_data(data, target_column)`

Nhận dữ liệu đã tiền xử lý và xác định cột mục tiêu `target_column` (trong project này là `'saturated_fat_g'`).

`split_data(test_size, stratify)`

Chia dữ liệu thành tập Train/Test theo tỉ lệ chỉ định, hỗ trợ `stratify` nếu cần. Giải phóng bộ nhớ data gốc sau khi split.

`set_training_data(train_processed, test_processed, target_col)`

Thiết lập dữ liệu đầu vào (X) và nhãn (y) cho quá trình huấn luyện từ các DataFrame đã xử lý. Tách X và y, giải phóng `train_df`, `test_df`.

`initialize_models()`

Khởi tạo danh sách các mô hình sẽ dùng: ElasticNet, RandomForest, LightGBM, XGBoost, DecisionTree.

`train_models(models_to_train)`

Thực hiện vòng lặp `fit` cho các mô hình được yêu cầu.

`optimize_params(model_name, n_trials, cv, n_jobs)`

Sử dụng Bayesian Optimization (thông qua Optuna) để tìm bộ siêu tham số tốt nhất cho một mô hình cụ thể.

`evaluate_models()`

Đánh giá toàn bộ các mô hình đã huấn luyện trên tập Test bằng `ModelEvaluator` và tự động xác định mô hình tốt nhất dựa trên R^2 .

`get_feature_importance(model_name, top_n)`

Lấy danh sách các đặc trưng quan trọng nhất ảnh hưởng đến kết quả dự đoán của mô hình đó. Nếu không chỉ định `model_name`, sử dụng `best_model`.

2.2.4 src/models/optimizer.py

Class: BayesianOptimizer

Class chuyên biệt thực hiện tối ưu hóa siêu tham số sử dụng thư viện Optuna. Định nghĩa sẵn không gian tìm kiếm (Search Space) cho từng loại model và sử dụng Bayesian Optimization để tìm kiếm tham số tối ưu.

Methods:

`__init__(X_train, y_train, random_state, cv, n_jobs)`

Khởi tạo Optimizer với dữ liệu huấn luyện, `random_state` cho reproducibility, số fold cross-validation `cv` và số jobs song song `n_jobs`.

`optimize(model_name, n_trials)`

Định nghĩa không gian tìm kiếm (search space) cho từng loại model

(RandomForest, LightGBM, XGBoost, ElasticNet, DecisionTree) và chạy quá trình tối ưu hóa với `direction='maximize'` để tối đa hóa R^2 trên tập validation.

2.3 Package: src.visualization

Package này cung cấp các công cụ trực quan hóa giúp hiểu rõ dữ liệu và kết quả mô hình thông qua các biểu đồ, gắn với các hình đã trình bày ở phần trước.

2.3.1 src/visualization/eda.py

Class: EDA

Class thực hiện *Exploratory Data Analysis* (Phân tích Dữ liệu Khám phá), giúp có cái nhìn tổng quan về dữ liệu trước khi mô hình hóa.

Methods:

`__init__(data, show_plots)`

Khởi tạo với dữ liệu cần phân tích và cờ `show_plots` để quyết định có hiển thị trực tiếp biểu đồ hay chỉ lưu file.

`summary_statistics(save_path)`

Thực hiện phân tích toàn diện: thông tin dataset, thống kê mô tả các cột số, phân tích giá trị khuyết (với barplot), phân tích dòng trùng lặp, tần suất giá trị và độ lệch (skewness) của dữ liệu.

`correlation_analysis(method, save_path)`

Tính toán ma trận tương quan và vẽ Heatmap. Hỗ trợ các phương pháp: 'pearson', 'spearman', 'kendall' hoặc 'all' để vẽ cả 3 loại, tương ứng với Hình 4, 5, 6.

`data_distribution(save_path)`

Vẽ biểu đồ histogram kết hợp đường cong mật độ (KDE) cho tất cả các cột số trên cùng một figure với nhiều subplots, như Hình 1.

`boxplot_analysis(save_path)`

Vẽ boxplot cho các đặc trưng số trên cùng một figure với nhiều subplots nhằm trực quan hóa ngoại lệ, như Hình 3.

`perform_eda(corr_method, save_path)`

Phương thức tiện ích chạy toàn bộ quy trình EDA theo trình tự chuẩn: `summary_statistics()`, `correlation_analysis()`, `data_distribution()`, `boxplot_analysis()`.

2.3.2 src/visualization/model_plots.py

Class: ModelVisualizer

Class chuyên biệt để trực quan hóa hiệu năng của các mô hình sau khi huấn luyện.

Methods:

`__init__(evaluation_results)`

Khởi tạo với kết quả đánh giá thu được từ `ModelTrainer.evaluate_models()` (dictionary chứa 'results' và 'best_model_name').

`plot_model_comparison(save_path)`

Vẽ 4 biểu đồ cột so sánh các chỉ số MSE, RMSE, MAE, R^2 giữa các mô hình trên cùng một figure, highlight model tốt nhất bằng màu khác, tương ứng với Hình 7.

`plot_feature_importance(importance_df, save_path)`

Vẽ biểu đồ thanh ngang thể hiện mức độ đóng góp của từng đặc trưng vào quyết định của mô hình. Tự động sắp xếp theo importance giảm dần, tương ứng với Hình 9.

3 Tổng quan thư viện và phương pháp tối ưu siêu tham số

3.1 Thư viện học máy sử dụng

Trong nghiên cứu này, bốn thư viện chính được sử dụng gồm **Scikit-learn (sklearn)**, **LightGBM**, **XGBoost** và **Optuna**.

Sklearn là thư viện phổ biến trong Python, cung cấp đầy đủ các thuật toán học máy truyền thống như hồi quy tuyến tính, cây quyết định và các phương pháp đánh giá mô hình.

LightGBM là thư viện chuyên dụng cho các mô hình cây tăng cường (Gradient Boosting), có tốc độ huấn luyện nhanh và hiệu quả cao với tập dữ liệu lớn.

XGBoost là thư viện học máy mạnh mẽ dựa trên cây quyết định, nổi tiếng với khả năng xử lý dữ liệu khuyết thiếu và tối ưu hóa hiệu suất tính toán.

Optuna là thư viện tối ưu siêu tham số tự động, hỗ trợ nhiều chiến lược tìm kiếm tiên tiến.

3.2 Các mô hình được sử dụng

Các mô hình hồi quy được áp dụng trong nghiên cứu gồm: **Elastic Net**, **Decision Tree Regression**, **Random Forest Regression**, **LightGBM Regression** và **XGBoost Regression**.

Elastic Net là mô hình tuyến tính kết hợp regularization L1 và L2, giúp phân tích mối quan hệ tuyến tính giữa biến đầu vào và đầu ra.

Các mô hình còn lại thuộc họ cây quyết định (tree-based) có khả năng mô hình hóa quan hệ phi tuyến phức tạp, trong đó **Random Forest**, **LightGBM** và **XGBoost** là các mô hình ensemble, được xây dựng ra cải thiện **Decision Tree**.

3.3 Chỉ số đánh giá mô hình

Hiệu quả của các mô hình được đánh giá thông qua ba chỉ số chính: **R-squared** (R^2), **Mean Squared Error** (MSE) và **Mean Absolute Error** (MAE).

- **Mean Squared Error (MSE)**: Công thức tính:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

MSE nên được sử dụng khi ta muốn phạt nặng các sai số lớn, tuy nhiên nó bị ảnh hưởng nếu xuất hiện giá trị ngoại lai.

- **Mean Absolute Error (MAE)**: Công thức tính:

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

MAE nên được sử dụng khi ta muốn đánh giá sai số tuyệt đối trung bình.

- **R-squared** (R^2): Công thức tính:

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

R^2 thể hiện khả năng giải thích biến động của các đặc trưng đầu vào đối với biến mục tiêu. Giá trị R^2 càng gần 1 thì mô hình càng tốt.

3.4 Tối ưu siêu tham số bằng phương pháp Bayesian

Quá trình tối ưu siêu tham số được thực hiện bằng phương pháp **Bayesian Optimization** thông qua thư viện Optuna. Phương pháp này xây dựng một mô hình xác suất cho hàm mục tiêu và lựa chọn bộ siêu tham số tối ưu dựa trên chiến lược cân bằng giữa khai thác và khám phá, giúp giảm đáng kể số lần thử nghiệm so với Grid Search và Random Search.

4 Kết quả thực nghiệm

Kết quả thực nghiệm chi tiết được tổng hợp trong file Google Sheets³. Cụ thể:

- **Sheet 1**: Giá trị trung bình các metrics cho từng cấu hình tiền xử lý và mô hình.
- **Sheet 2**: Tên mô hình cho ra kết quả tốt nhất tương ứng với từng phương pháp xử lý dữ liệu.
- **Sheet 3**: Toàn bộ kết quả của tất cả mô hình ở các lần chạy.
- **Sheet 4**: Sắp xếp các kết quả ở Sheet 1 theo R^2 giảm dần.

³<https://docs.google.com/spreadsheets/d/1DugDKIYezP1DGX1fH01LJ1cdBhQisnCArAE03LLE0dM/edit?usp=sharing>

- **Sheet 5:** Sắp xếp toàn bộ kết quả ở Sheet 3 tăng dần, để xác định đâu là phương pháp kém hoặc mô hình kém.

Dựa trên kết quả trung bình các metrics, nhóm tiến hành xếp hạng các cấu hình tiền xử lý và thu được **top 5** phương pháp có kết quả trung bình tốt nhất như trong Bảng 2.

num_strategy	scaling_strategy	outlier_method	r2_score
drop	standard	zscore	0.762583344
median	standard	zscore	0.762583344
mean	standard	zscore	0.760899558
mode	standard	isolation_forest	0.759473055
drop	standard	isolation_forest	0.759182476

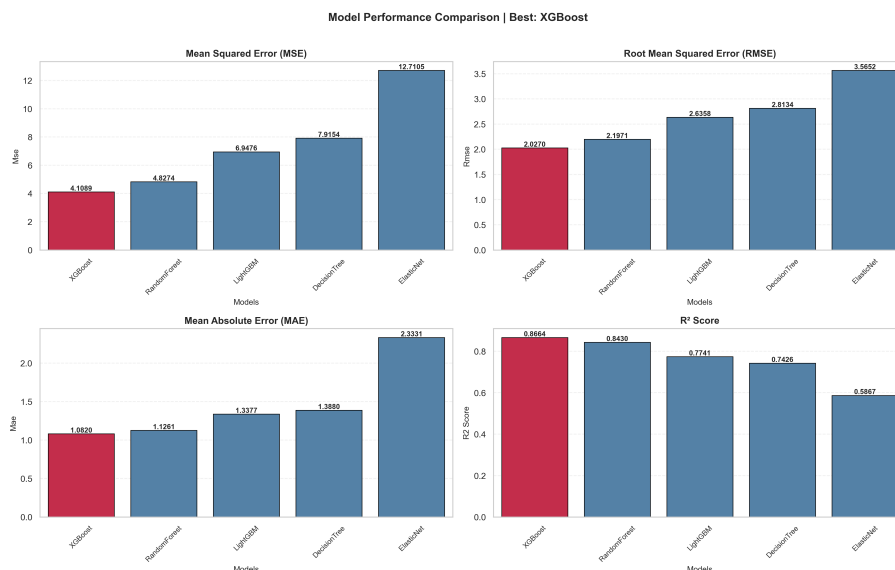
Bảng 2: Top 5 cấu hình tiền xử lý dữ liệu có kết quả trung bình tốt nhất

Dựa vào Sheet 5, ta nhận thấy mô hình Elastic Net luôn cho ra kết quả tệ nhất có thể ảnh hưởng tới các phương pháp được lựa chọn. Ngoài ra việc loại bỏ outlier bằng phương pháp IQR cho ra kết quả không tốt có thể do việc loại bỏ nhiều outlier hơn (so với zscore) dẫn tới mất một số thông tin quan trọng trong dữ liệu.

Do đó, ở các bước tiếp theo, nhóm lựa chọn cấu hình tiền xử lý gồm: loại bỏ giá trị khuyết (**drop**), chuẩn hoá **standard**, và xử lý ngoại lệ bằng **zscore** để tiến hành trực quan các kết quả⁴.

4.1 So sánh các mô hình học máy

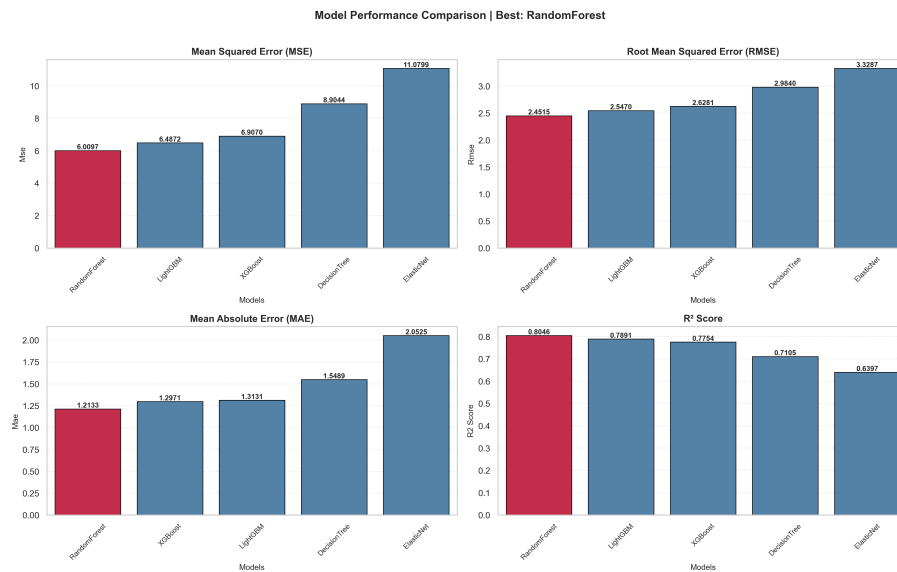
Sau khi cố định quy trình tiền xử lý như trên, nhóm tiến hành tối ưu siêu tham số, huấn luyện và đánh giá **5 mô hình** khác nhau. Kết quả so sánh các mô hình dựa trên các metrics đánh giá được biểu diễn trong Hình 7.



Hình 7: Kết quả so sánh các mô hình học máy trước khi tối ưu siêu tham số

⁴Ở đây không thực hiện tối ưu siêu tham số vì cho ra kết quả tệ hơn khi chưa sử dụng tối ưu, nguyên nhân có thể do overfitting quan sát hai hình 7 và 8

Quan sát Hình 7, **XGBoost Regression** là mô hình có hiệu năng tốt nhất trên cả 4 metrics, với R^2 đạt 0.8664 có thể nói rằng mô hình này dự đoán khá tốt với bộ dữ liệu hiện tại.

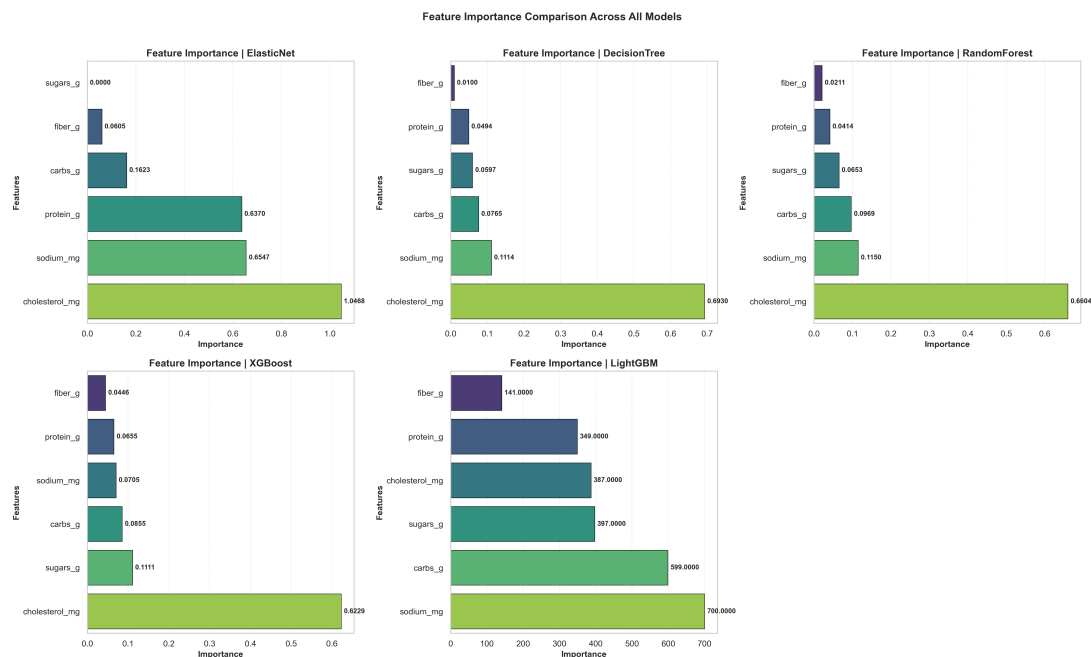


Hình 8: Kết quả so sánh các mô hình học máy sau khi tối ưu siêu tham số

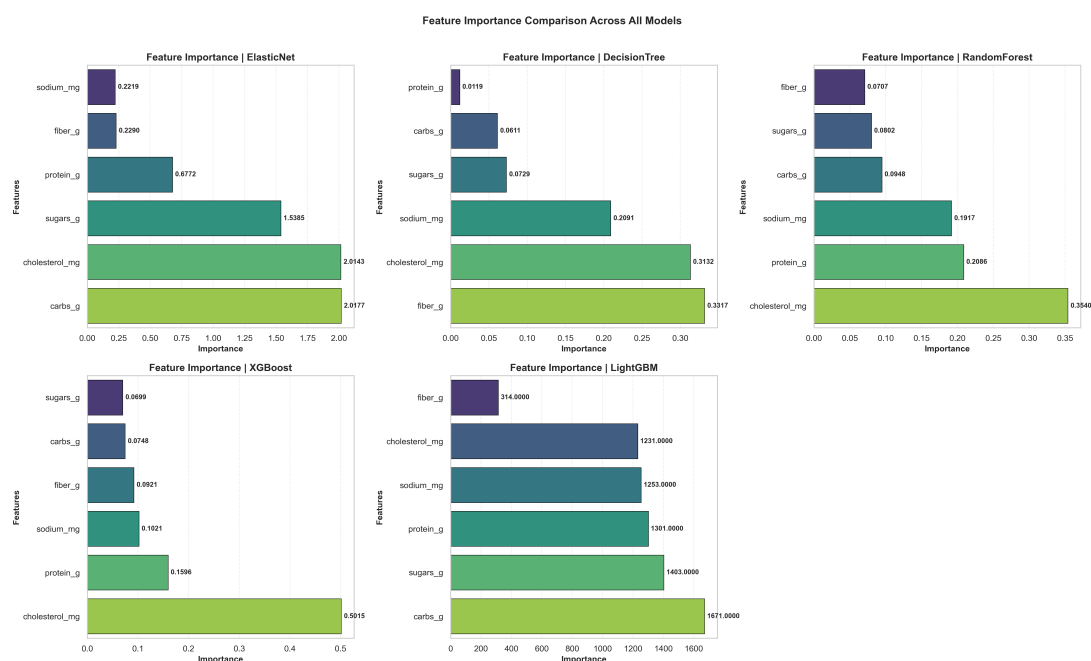
Khi thực hiện tối ưu siêu tham số với hàm mục tiêu là tối đa hoá R^2 , ta thu được kết quả như trong Hình 8. Chỉ có **LightGBM Regression** và **ElasticNet** là cải thiện hiệu năng, trong khi các mô hình còn lại đều giảm. Đáng kể nhất là **XGBoost Regression** giảm từ 0.8664 xuống còn 0.7754.

4.2 Phân tích đặc trưng quan trọng của các mô hình

Kết quả trực quan hoá các đặc trưng quan trọng được thể hiện trong Hình 9.



Hình 9: Các đặc trưng quan trọng theo đề xuất của 5 mô hình trước khi tối ưu



Hình 10: Các đặc trưng quan trọng theo đề xuất của 5 mô hình sau khi tối ưu

Từ Hình 9, có tới 4 mô hình đồng thuận rằng các đặc trưng **cholesterol_mg** là yếu tố quan trọng nhất ảnh hưởng đến hàm lượng **saturated_fat_g**.

Mặc dù **sugars_g** không có độ tương quan cao với mục tiêu, trái lại được mô hình XGBoost đánh giá là quan trọng gấp 3 lần so với **fiber_g** là đặc trưng có tương quan mạnh khi chúng ta phân tích ở Hình 4, Hình 5 và Hình 6. Có thể mô hình XGBoost đã phát hiện ra mối quan hệ giữa **sugars_g** và **saturated_fat_g** do đó nó có kết quả tốt nhất, điều này càng củng cố hơn khi ta phân tích đặc trưng ở Hình 10, **sugars_g** được đánh giá khá thấp hơn dẫn tới kết quả tệ.

5 Bảng công việc

Tên	MSSV	Công việc
Nguyễn Hoàng Anh Duy	23280051	- Xây dựng phần tiền xử lí: DataPreprocessor - Xây dựng phần EDA: Visualization/EDA - Viết pipeline EDA - Làm slide thuyết trình - Bổ sung tham số (argparse) - Chỉnh sửa docstrings và annotations - Hỗ trợ chạy thực nghiệm
Nguyễn Hải Lâm	23280068	- Xây dựng phần mô hình: ModelTrainer - Xây dựng phần trực quan kết quả: ModelVisualizer - Viết pipeline huấn luyện, tối ưu tham số - Giúp pipeline hỗ trợ truyền config và tham số linh hoạt (argparse) - Chỉnh sửa các phương thức, đảm bảo nguyên tắc vàng - Chỉnh sửa slide thuyết trình
Phạm Khang Bình	23280043	- Tổng hợp, tổ chức lại các thuộc tính, phương thức và lớp - Tổ chức mã nguồn phù hợp - Chỉnh sửa docstrings và annotations - Viết pipeline cho toàn bộ chương trình, phần thực nghiệm - Viết báo cáo